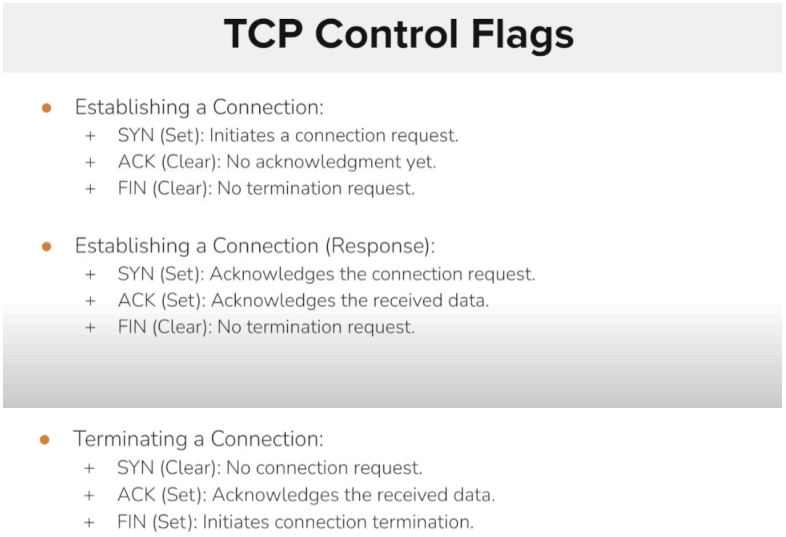
EJPT - Section 2: Footprinting & Scanning

Elías Vallina
Spring 2026, Galway, Ireland

TCP 3-WAY-HANDSHAKE



`netstat` (abreviatura de network statistics) es un comando que se usa para mostrar información detallada sobre la red de tu máquina.

```
netstat -antp
```

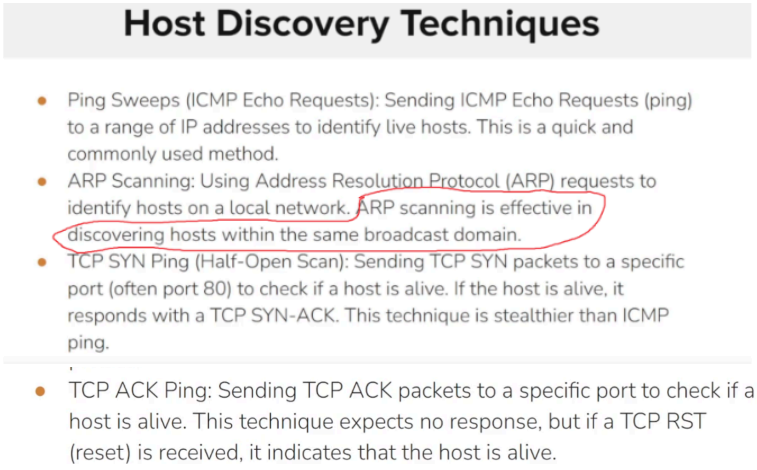
`-a` (all): Muestra todas las conexiones y puertos, incluyendo:

- Conexiones activas (established)
- Puertos en escucha (listening)
- Conexiones en otros estados (SYN_SENT, TIME_WAIT, etc.)

`-n` (numeric): Muestra direcciones IP y números de puerto en formato numérico (sin resolver nombres de host ni nombres de servicios). Ejemplo: en vez de "localhost:http" muestra "127.0.0.1:80". -> Esto hace que el comando sea más rápido y evita consultas DNS innecesarias.

`-t` (tcp): Limita la salida solo a conexiones TCP (ignora UDP, raw, unix sockets, etc.).

`-p` (program): Muestra el PID (Process ID) y el nombre del programa/proceso que está usando cada socket/conexión.



¿Qué pasa cuando mando un paquete ACK "falso"?

- Si el host está vivo (existe y tiene IP activa):
  - El sistema operativo del host ve el paquete ACK.
  - Se da cuenta de que no hay ninguna conexión previa con ese número de secuencia.
  - Entonces responde automáticamente con un paquete RST (Reset) → "¡Error! No conozco esta conexión, reseteo todo".
  - Recibir un RST = el host está vivo (porque alguien respondió).
- Si el host no está vivo (apagado, sin IP, o filtrado por firewall):
  - Nadie responde nada.
  - No hay respuesta = Nmap asume que podría estar down (o muy filtrado).

1. SYN ← Yo (cliente) inicias
2. SYN-ACK ← Servidor responde
3. ACK ← Tú confirmas

The same can be also can be done with a TCP SYN–ACK packet! :

SYN-ACK Ping (Sends SYN-ACK packets): Sending TCP SYN-ACK packets to a specific port to check if a host is alive. If a TCP RST is received, it indicates that the host is alive.

Also bear this in mind:

The choice of the "best" host discovery technique in penetration testing depends on various factors, and there isn't a one-size-fits-all answer.

Pros and Cons

- ICMP Ping:
  - Pros: ICMP ping is a widely supported and quick method for identifying live hosts.
  - Cons: Some hosts or firewalls may be configured to block ICMP traffic, limiting its effectiveness. ICMP ping can also be easily detected.
- TCP SYN Ping:
  - Pros: TCP SYN ping is stealthier than ICMP and may bypass firewalls that allow outbound connections.
  - Cons: Some hosts may not respond to TCP SYN requests, and the results can be affected by firewalls and security devices.

Ping sweeps:

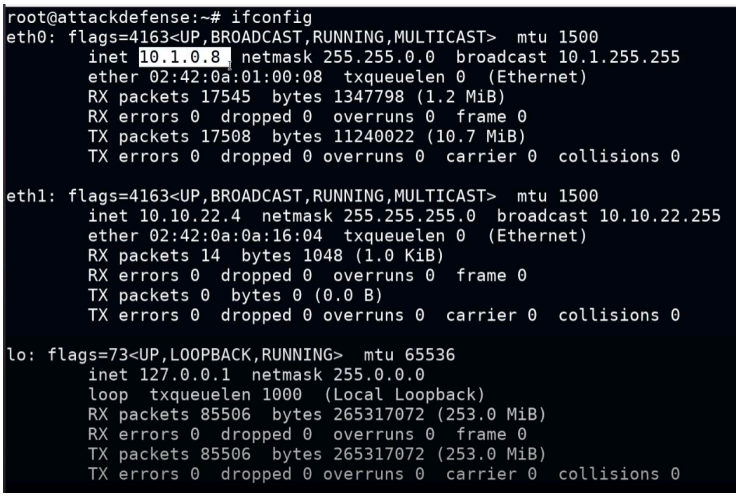
So, when a device sends an ICMP Echo Request, it creates an ICMP packet with Type 8, Code 0.

When the destination device receives the Echo Request and responds with an Echo Reply, it creates an ICMP packet with Type 0, Code 0.

- ICMP Echo Request:
  - Type: 8
  - Code: 0
- ICMP Echo Reply:
  - Type: 0
  - Code: 0

Important: Windows by default doesn't reply an ICMP Echo Request!

LAB



eth0 la red “de gestión” (para que tengamos acceso al entorno, NAT, servicios internos del proveedor, etc.)

eth1 la red del laboratorio, donde están las máquinas objetivo.

Como lo deducimos:

1-
eth0: red grande (/16) + mucho tráfico → típico de gestión
eth1: red /24 + poco tráfico → típico de red de objetivos (pentesting que haremos)

2-
eth0 tiene muchísimos paquetes (17k+) → está muy activa (NAT/servicios/tu sesión)
eth1 casi no tiene tráfico → típico de una red donde solo hay tráfico cuando se escanea o ataca.

Nota: los 17k y los 4 paquetes RX: es el total desde que se levantó la interfaz (normalmente desde que se arrancó la máquina).

```
root@attackdefense:~# ping -b -c 1 10.10.23.0
```

Al hacer el comando que tenemos encima, al poner .0, indicamos que queremos analizar la subnet entera.

fping mejor:

```
root@attackdefense:~# fping -a -g 10.10.23.0/24
```

¿Porqué? fping tiene la opcion -g que genera todas las Ips del rango , en este caso del rango /24 de esta subred. -a muestra solo las que responden (host alive).
Es decir, como resultado hace un ping sweep de toda la subred y lista los que responden.

Si intentara hacer con ping en vez de fping:
ping no tiene la opcion -g , no genera rangos sino que la opción ping solo prueba una IP por comando.

```
root@attackdefense:~# fping -a 10.4.31.111
```

: hace ping ICMP SOLO! y, con -a, solo imprime la IP si responde. Si ICMP está bloqueado, puede parecer "muerto" aunque esté vivo

vs

```
root@attackdefense:~# nmap -sn 10.4.31.111
```

: Unicamente hace ping ICMP pero si no se obtiene respuesta, puede usar ARP (en LAN) y/o probes TCP/otros según el caso.

HOST DISCOVERY WITH NMAP

-sn only attempts to host discovery, not port scanning!
Misma LAN: nmap -sn usa ARP (no ICMP) porque es lo más fiable.
Red remota: nmap -sn usa ICMP Echo y puede combinarlo con TCP probes (SYN/ACK).

LAN = Local Area Network == A local network in a small area (home/office/lab)
ARP only works in a LAN. It cannot go through routers.

Para abrir wireshark:

```
root@attackdefense:~# sudo wireshark -i eth1
```

Hacemos:

```
root@attackdefense:~# nmap -sn 10.1.0.0/24
```

: hace un descubrimiento de hosts ("ping scan") en toda la subred /24 (256 IP's)

A veces habrá que darle privilegios para q funcione! Si soy root o sudo, Nmap combina ICMP y TCP! -> más posibilidad de obtener respuesta de los hosts de la target network!

-ns lo unico que nos dice es : no escanear puertos, únicamente dar lista de hosts vivos (que responden).

Sin embargo al ver en wireshark, solo aparecen ARP paquetes. Esto pasa porq el ordenador del instructor esta conectado por ethernet, y si estas conectado por ethernet se enviar ARPs. Para anularlo (queremos ICMP, ya que nmap está pensado para hacer ICMP, era la intención al usarlo) hacemos:

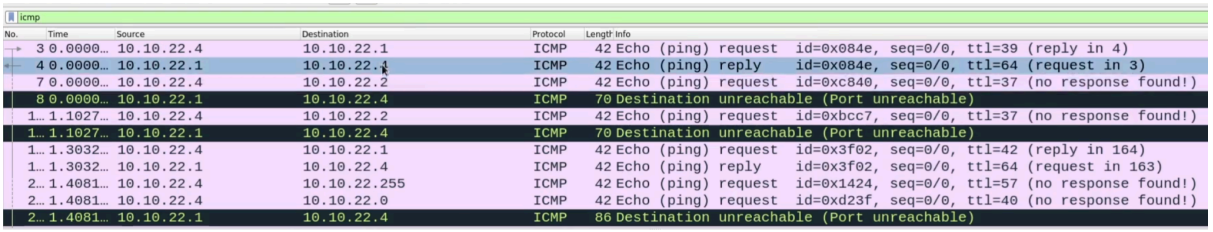
```
root@attackdefense:~# nmap -sn 10.10.22.0/24 --send-ip
```

El resultado es:

```
Nmap done: 256 IP addresses (2 hosts up) scanned in 20.31 seconds
root@attackdefense:~#
```

Esos 2 hosts son 10.10.22.1 y 10.10.22.1 (soy yo el .1, mi kali linux IP !!!)
Al hacer nmap siempre saldrá mi IP en la lista de hosts up!

En wireshark vemos:



Para escanear varias Ips que nos interesen:

```
root@attackdefense:~# nmap -sn 10.4.23.227 10.4.23.228
```

Para escanear un rango de Ips:

```
root@attackdefense:~# nmap -sn 10.4.23.227-240
```

Para hacerlo más comodo si tenemos un rango determinado de IPS: crear un documento .txt :

```
root@attackdefense:~# vim targets.txt
```

Escribimos por ejemplo las target Ip, y le damos a -wq (write and quit)

Ahora para hacer nmap en estas Ips en concreto del .txt hacemos:

```
root@attackdefense:~# nmap -sn -iL targets.txt
```

-PS option: overrides all of the -sn options with regards to what type of packets are being sent. with -PS option set, now the packet sent will be a SYN packet instead of ARP's (bc ethernet). Look below , see:

```
root@attackdefense:~# nmap -sn 10.4.23.227
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 15:07 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.08 seconds
root@attackdefense:~# nmap -sn -PS 10.4.23.227
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 15:08 IST
Nmap scan report for 10.4.23.227
Host is up (0.0083s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds
root@attackdefense:~#
```

As observed, thanks to the -PS option we can detect one host!

When observing wireshark:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000…	10.10.22.4	10.4.23.227	TCP	58	50560 → 80 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2	0.0082…	10.4.23.227	10.10.22.4	TCP	58	80 → 50560 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=
3	0.0082…	10.10.22.4	10.4.23.227	TCP	54	50560 → 80 [RST] Seq=1 Win=0 Len=0

The third row, instead of a ACK (the 3rd in three way handshake in TCP), we see it's an RST packet. RST =1 means : abort this connection right now, this does not continue anymore. The reason for RST is just because we don't wanna stablish any connection, we only wanna receive any packet from the target host!

Host discovery with ports technique

By default, when sending a SYN Ping, (-PS option), it is sent to port 80. If we wanna change that, just type the port next to the PS , in this case the 22:

```
root@attackdefense:~# nmap -sn -PS22 10.4.23.227
```

Si queremos poner un rango de puertos en vez de únicamente el 22:

```
root@attackdefense:~# nmap -sn -PS1-1000 10.4.23.227
```

Si son unos cuantos concretos:

```
root@attackdefense:~# nmap -sn -PS80,3389,445 10.4.23.227
```

When performing a host discovery , don't do a port scan yet!!

When the hosts alive have been identified, it's time to perform a port scan!
nmap host discovery , sending ACK packet corresponding to TCP protocol

Not a very reliable technique to use, as many devices have blocked the packets with the enabled (ACK flag=1)

```
root@attackdefense:~# nmap -sn -PA 10.4.23.227
```

nmap host discovery, forcing ICMP echo request packets

```
root@attackdefense:~# nmap -sn -PE 10.4.23.227
```

BEST METHOD according to the instructor

```
root@attackdefense:~# nmap -sn -v -T4 10.4.23.227
```

-v : verbose: displays more output (information) on the screen
-T4 : to increase the speed of the scan or the number of packets being sent.
- in the end, we need to put the IP's we wanna scan

Below, note we can also combine methods in the same command (forced SYN for 21,22,... but this doesn't mean maybe the nmap command will try some other ports with a protocol different than TCP SYN.

```
root@attackdefense:~# nmap -sn -PS21,22,25,80,445,3389,8080 -T4 10.4.20.217
```

Nmap host discovery

Nmap has 2 phases:

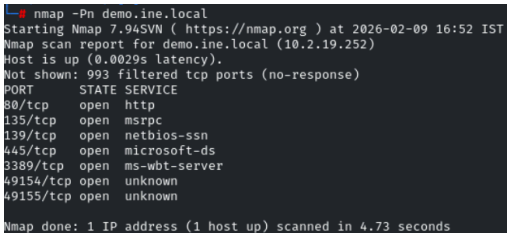
1-host discovery
2- port scan

By default in nmap, from the hosts didn't receive a reply, nmap won't perform phase 2 (assumes is dead).

The following command :

```
nmap -Pn demo.ine.local
```

is saying: skip phase 1 (host discovery) and goes straight to phase 2 (port scan) , since : even if you already specify the IP, Nmap still performs host discovery by default: it first tries to verify whether that host is alive : (-Pn option!!!)
This is the result:

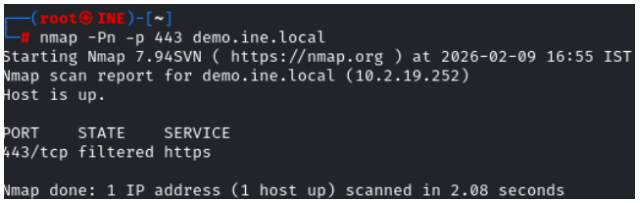


In the capture above, that doesn't mean the ports displayed in it are the only ones scanned! Many more ports were scanned, specifically 1000 ports by defafult (the most common TCP ports), but only the open ones are displayed

If we wanna scan any port that is not displayed in the capture above, we perform:

```
nmap -Pn -p 443 demo.ine.local
```

The output below:



"Filtered" does not mean "not open": it means Nmap could not determine the port's state because there was no response, likely due to a firewall.

If there's not firewall, nmap will be able to say if it's closed:

Note: Filters (as firewalls) often drop probes silently, so Nmap retries (more attempts) to rule out packet loss (descartar que sea pérdida de paquetes), which greatly slows the scan (more seconds will take the process).

If we wanna identify the service and its version in a port, use -sV as follows:

Important: In addition to this data: service name and its version, sometimes during the interaction between our host and the target, the latter responds by providing additional data such as the hostname, the workgroup name, as well as extra domain information.

For the exam, keep in mind: If a question asks for details about a server, check the service running on its ports with -sV, because the service may reveal server information such as software, version, hostname, or workgroup.

Example to see the relationship between server & service:

server = the target machine (for example a super computer) running the Samba software.

service = the server who runs Samba software listening on port 445

```
nmap -Pn -sV -p 80 demo.ine.local
```

Below is The resulting output:

```
PORT   STATE SERVICE VERSION
80/tcp open  http    HttpFileServer httpd 2.3
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

Service: http (protocol); Application: httpFileServer (program listening on that port and attending the connexions); Version: 2.3

PORT SCANNING

TCP PORTS

2 distinctions:

(1)nmap with no sudo or root privileges: perfoms the 3 steps of the 3-way-handshake (-sT default option, this means 1- SYN, 2- SYN/ACK // if its closed it will send a RST, 3- ACK). It will also appear in the 99% of cases a 4th paquet (me) to abort the connection (usually 4- FIN/ACK or RST).
(2)nmap with root /sudo privileges on : starts with a SYN but does not complete the 3-way handshake (SYN, SYN ACK or RST (if target closed), RST).
Note: Regardless, if we want a SYN SCAN, better specify it with -sS. It is stealthier!

RST or similar (the end-connection packet) is needed to send just because we're in TCP protocol, so it is non-negotiable!

If we don't receive neither SYN-ACK nor RST (second paquet), the most probably is there's a firewall and probaby that port was actually opened.

TCP SYN SCAN (2) vs TCP CONNECT SCAN (1)
Tcp connect scan isn't the preferred bc is too loud. However, is more reliable (mainly bc of the third packet (here's the difference between (1) and (2), in which (1) are easily to detect by firewalls because its configuration) and accurate bc it completes the three way handshake.

nmap fast scan : reduces from 1000 to 100 the ports scanned (the most common ones):

```
root@attackdefense:~# nmap -Pn -F 10.4.24.205
```

Note thanks to -F is executed the fast scan.

Below, we do a scan to the entire port TCP range (-p-).

```
root@attackdefense:~# nmap -Pn -p- 10.4.24.205
```

```
6 0.009… 10.10.23.4        10.4.24.205      TCP   54 43842 → 80 [RST] Seq=1 Win=0 Len=0
```

in the capture above, this is a syn ack tcp packet! Not a Ack packet (the last in the 3-way-handshake).

UDP PORTS

just we have to add -sU :

```
root@attackdefense:~# nmap -Pn -sU 10.4.24.205
```

Important: UDP scan is very slow, so don't perform -p- -sU, it will never end xd. Better specify a range or the 2000 most common ports, as instance.

-A option (Agressive):

Without -A: only checks the port state (TCP by default) → e.g., 177/tcp open|closed|filtered

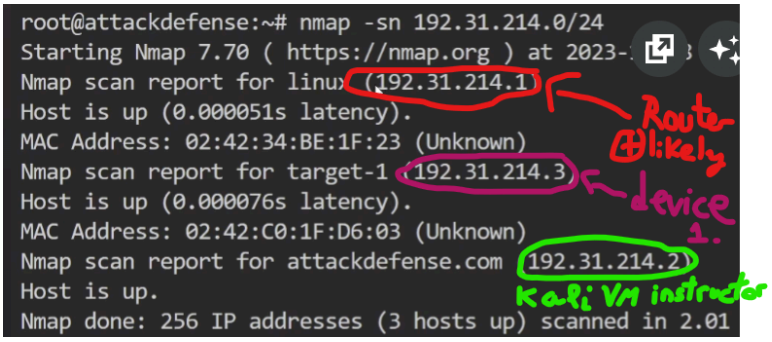
With -A: port state + extra enumeration (-sV service/version, -sC default scripts, -O OS guess, --traceroute) → slower, noisier, easier to detect.

Detection of Service Version and OS

Despite The PC's Instructor IP is 192.31.214.2, he starts performing a nmap scan to the entire subnet range:

```
root@attackdefense:~# nmap -sn 192.31.214.0/24
```

The output is as follows:



When performing:

```
root@attackdefense:~# nmap -T4 -sS -sV  -p- 192.31.214.3
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 16:37 UTC
Nmap scan report for target-1 (192.31.214.3)
Host is up (0.0000090s latency).
Not shown: 65532 closed ports
PORT      STATE SERVICE   VERSION
6421/tcp  open  mongodb   MongoDB 2.6.10
41288/tcp open  memcached Memcached
55413/tcp open  ftp       vsftpd 3.0.3
MAC Address: 02:42:C0:1F:D6:03 (Unknown)
Service Info: OS: Unix
```

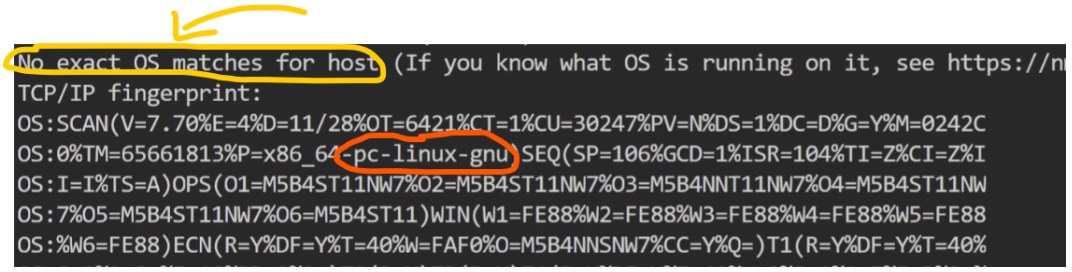
OS: Unix means that the most likely OS in the target host is Unix. This statement is made based on the way the host responds to our command (nmap -T4 -sS -sV -p- 192.31.214.3).

The question comes up at this point is: how can we possibly determine what OS is running on that system (host) ?

Try:
-O: operating system

```
root@attackdefense:~# nmap -T4 -sS -sV -O -p- 192.31.214.3
```

The output is:



In this manner, we conclude that the command is not able to figure out what's the OS, but is more likely to be related to linux.

Nevertheless, there's an other command more aggressive and that focuses on determine the OS, with the -osscan-guess :

```
root@attackdefense:~# nmap -T4 -sS -sV -O --osscan-guess -p- 192.31.214.3
```

the output is:

```
Aggressive OS guesses: Linux 2.6.32 (96%), Linux 3.2 - 4.9 (96%), Linux 2.6.32 - 3.10 (96%), Linux 3.4 - 3.
0 (95%), Synology DiskStation Manager 5.2-5644 (95%), Linux 3.1 (95%), Linux 3.2 (95%), AXIS 210A or 211 Ne
```

So below are displayed the most likely OS ,

```
Aggressive OS guesses: Linux 2.6.32 (96%), Linux 3.2 - 4.9 (96%), Linux 2.6.32 - 3.10 (96%), Linux 3.4
0 (95%), Synology DiskStation Manager 5.2-5644 (95%), Linux 3.1 (95%), Linux 3.2 (95%), AXIS 210A or 21
```

They're not probabilities: they're similarity percentages to Nmap's fingerprints.
Several can be 96% because many OS behave similarly, and NAT/VMs/firewalls blur the fingerprint

Another option to be more sure about the reliability of the %, do:

```
nmap -T4 -sS -sV --version-intensity 8 -O --osscan-guess -p- 192.31.214.3
```

–version-intensity requires a mark from 0 to 9 , where 9 will provide the most reliable/accurate % about the most likely OS.

Nmap scripting engine (NSE)

A normal scan tells you: *open/closed/filtered* and maybe the service/version.

NSE scripts let Nmap interact with the service to pull more info or test things.

Default nmap scan

El default Nmap script scan (-sC) sirve para sacar información extra automáticamente de los servicios que encuentras abiertos, sin escanear más puertos.

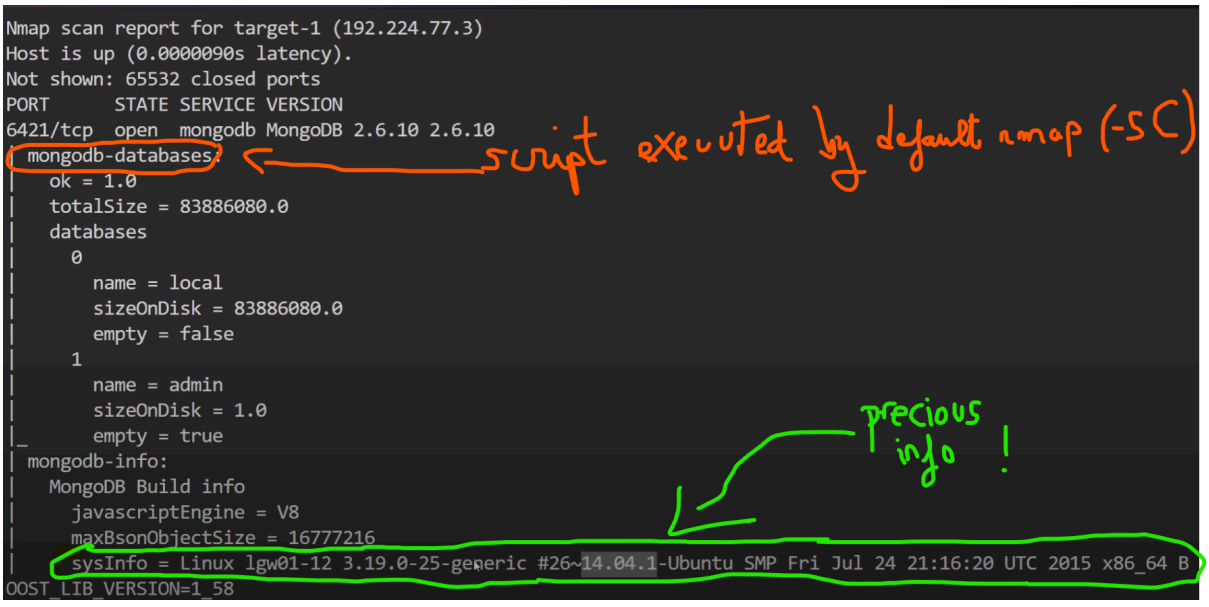
En vez de solo decir "el puerto 80 está abierto", intenta averiguar cosas como:

- qué web/servicio es (título, banners),
- headers y config básica (HTTP/HTTPS, certificado),
- detalles típicos de servicios (SSH/SMB/DNS, etc.).

Es como un "escaneo básico inteligente" para identificar mejor qué hay detrás de cada puerto.

```
root@attackdefense:~# nmap -sS -sV -sC -p- -T4 192.224.77.3
```

When running the following command, the result is :



The green circle is just what we were seeking! Also, provides extra information apart from the OS (between the orange and green circles).

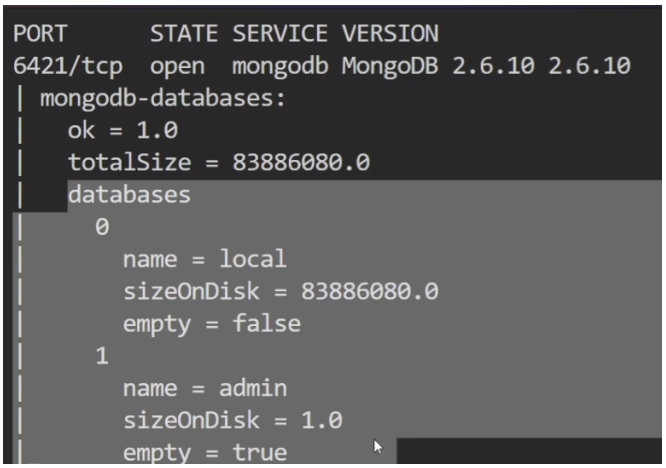
The orange circle, shows the script the -sC option executed. To understand better:

Nmap detecta el servicio/versión del puerto (en tu salida: MongoDB 2.6.10). Al ver "esto parece MongoDB", Nmap mira sus scripts NSE y ejecuta los que:

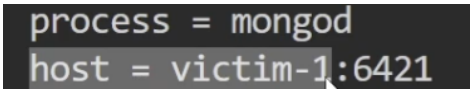
- *están en la categoría default (por -sC), y*
- *tienen una regla de puerto/servicio que encaja (por ejemplo, scripts mongodb-* que se lanzan cuando detecta MongoDB).*

According to this 2 points, in this case, the script executed is mongodb-dataset.

As mentioned, between the 2 circles we have very important information, the selected section below:

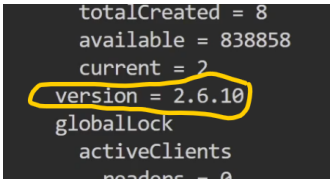


As displayed in the image above, we have the specific name of the databases, 2 specifically:local and admin, and we can see one is no empty and the second one it is.



This is also very important information. The host name of the operating system: victim-1 . For example in my PC the host name is ELIAS VALLINA. 6421 as known previously, is the port where the MongoDB service is running.

Now whe're this point, all this information is extremely useful to us. Why? For example, we can start searching for vulnerabilities that affect this version of MongoDB, or other example, start looking for vulnerabilities that may affect the OS (in this case our target is ubuntu system as shown before in the green circle).



```
root@attackdefense:~# ls -al /usr/share/nmap/scripts/ | grep -e "mongodb"
```

in the directory above are stored all the nmap scripts! To search among all of them for those that have a specific name (in the image, 'mongodb'), you do it like this with grep -e.

The output to this command are 3 scripts:

```
-rw-r--r-- 1 root root  2578 Jan  9  2019 mongodb-brute.nse
-rw-r--r-- 1 root root  2583 Jan  9  2019 mongodb-databases.nse
-rw-r--r-- 1 root root  3663 Jan  9  2019 mongodb-info.nse
```

In the following command we see that 2 of this 3 scripts are indeed part of the default ones for the MongoDB scripts:

```
root@attackdefense:~# nmap --script-help=mongodb-databases
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 18:26 UTC

mongodb-databases
Categories: default discovery safe
https://nmap.org/nsedoc/scripts/mongodb-databases.html
  Attempts to get a list of tables from a MongoDB database.
root@attackdefense:~# nmap --script-help=mongodb-info
Starting Nmap 7.70 ( https://nmap.org ) at 2023-11-28 18:27 UTC

mongodb-info
Categories: default discovery safe
https://nmap.org/nsedoc/scripts/mongodb-info.html
  Attempts to get build info and server status from a MongoDB database.
```

If we wanna execute a particular script, not the -sC mode that will choose one of the default ones, we do as follows. In this example we wanna execute the mongodb-info script:

```
root@attackdefense:~# nmap -sS -sV --script=mongodb-info -p- -T4 192.224.77.3
```

to do an aggressive scan (-A) (faster scan), but less stealth, do:

```
root@attackdefense:~# nmap -sS -A -p- -T4 192.224.77.3
```

LAB : SCAN THE SERVER I

objective: using Nmap, perform:

1- port scanning

2- service detection

target: demo.ine.local

step 0 (preliminar, not important in this lab)

to se the interfaces, i perform the following command:

```
┌──(root㉿INE)-[~]
└─# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
       valid_lft forever preferred_lft forever
2: ip_vti0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN group default qlen 1000
    link/ipip 0.0.0.0 brd 0.0.0.0
719242: eth0@if719243: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:0a:01:00:04 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.1.0.4/16 brd 10.1.255.255 scope global eth0
       valid_lft forever preferred_lft forever
719245: eth1@if719246: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:c0:5e:8b:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.94.139.2/24 brd 192.94.139.255 scope global eth1
       valid_lft forever preferred_lft forever
```

there are 3 active interfaces (4 in total, but one is DOWN (as if it didn't exist))

to know what is the interface that interests us in this lab (the one which interacts with the target) , we perform:

trick for the second step:

```
┌──(root㉿INE)-[~]
└─# ping -c 4 demo.ine.local
PING demo.ine.local (192.94.139.3) 56(84) bytes of data.
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=1 ttl=64 time=0.119 ms
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=2 ttl=64 time=0.053 ms
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=3 ttl=64 time=0.062 ms
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=4 ttl=64 time=0.056 ms

── demo.ine.local ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3056ms
rtt min/avg/max/mdev = 0.053/0.072/0.119/0.027 ms

┌──(root㉿INE)-[~]
└─# ip route get 192.94.139.3
192.94.139.3 dev eth1 src 192.94.139.2 uid 0
    cache
```

as observed, first we discover the ip in number format. why? bc ip route get doesn't work with a ip in letter format (demo.ine.local).

Secondly, and the objective of this explanation, we type ip route get ip_direction in order to discover what is the interface that communicates with the target.

as shown in the screenshot above, is on eth1, this is, 192.94.139.2.

step 1 (already done) : verify the target is reachable

```
┌──(root㉿INE)-[~]
└─# ping -c 4 demo.ine.local
PING demo.ine.local (192.94.139.3) 56(84) bytes of data.
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=1 ttl=64 time=0.119 ms
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=2 ttl=64 time=0.053 ms
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=3 ttl=64 time=0.062 ms
64 bytes from demo.ine.local (192.94.139.3): icmp_seq=4 ttl=64 time=0.056 ms

── demo.ine.local ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3056ms
rtt min/avg/max/mdev = 0.053/0.072/0.119/0.027 ms
```

step 2: nmap

I perform the following command,

```
└─# nmap demo.ine.local
```

but as by default nmap scans only the 1000 most common ports, we don't get any opened port.

to detect if any of the around 65k ports remain open, we must scan throughout range, so I add the -p- option:

```
└─# nmap demo.ine.local -p-
```

the output is:

```
└─# nmap demo.ine.local -p-
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-23 15:26 IST
Nmap scan report for demo.ine.local (192.195.101.3)
Host is up (0.000029s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
6421/tcp  open  nim-wan
41288/tcp open  unknown
55413/tcp open  unknown
MAC Address: 02:42:C0:C3:65:03 (Unknown)
```

Once all range scanned, and as shown above, 3 ports are detected.

With the following command, we'll analyze closely that 3 ports with a SYN scan (default when root permissions):

LAB : SCAN THE SERVER II

The objectives of this lab are to:

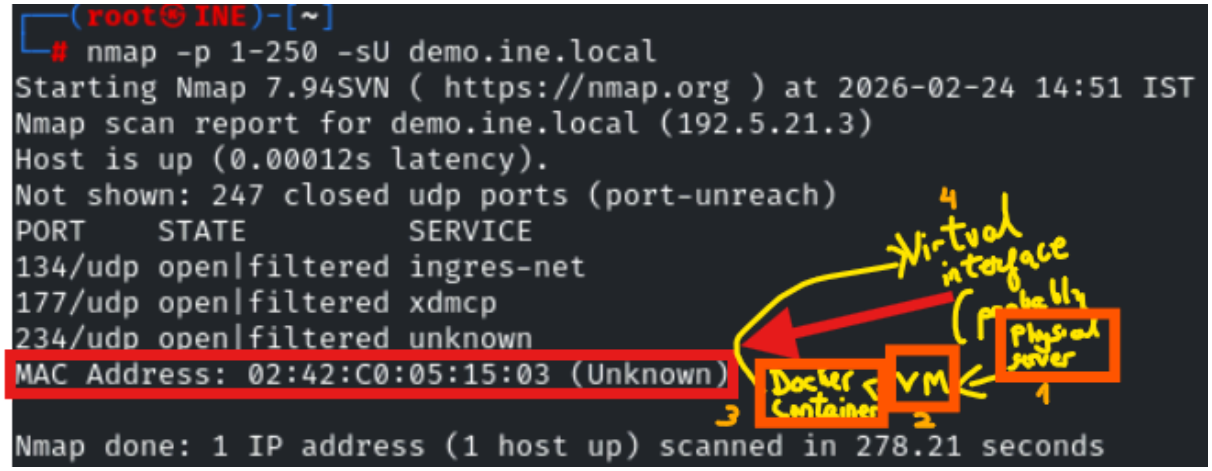
- Identify the port running a Bind DNS server.
- Identify the port running a TFTP server.
- Identify the port running the SNMP server.

if we google it, we see this 3 services are UDP. Also we observe the default ports of this 3 services are low, among the range from 1-250.

considering that, I perform the following command within the mentioned range, just because if i scan the entirely range tcp (-p-), it takes so much time and is unnecessary scan all the range in this case, as we know the rang:

```
┌──(root㉿INE)-[~]
└─# nmap -p 1-250 -sU demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-24 14:51 IST
Nmap scan report for demo.ine.local (192.5.21.3)
Host is up (0.00012s latency).
Not shown: 247 closed udp ports (port-unreach)
PORT    STATE         SERVICE
134/udp open|filtered ingres-net
177/udp open|filtered xdmcp
234/udp open|filtered unknown
MAC Address: 02:42:C0:05:15:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 278.21 seconds
```



Next, given that we know the ports number, we're gonna perform a detailed scan:

```
┌──(root㉿INE)-[~]
└─# nmap demo.ine.local -p 134,177,234 -sU -sV
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-25 13:10 IST
Nmap scan report for demo.ine.local (192.127.27.3)
Host is up (0.000050s latency).

PORT    STATE         SERVICE    VERSION
134/udp open|filtered ingres-net
177/udp open          domain     ISC BIND 9.10.3-P4 (Ubuntu Linux)
234/udp open          snmp       SNMPv1 server; net-snmp SNMPv3 server (public)
MAC Address: 02:42:C0:7F:1B:03 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

as shown above, we get very valuable information at port 177 and 234, but not on 134.

Also the -sV reveals us the OS is Linux (last line).

In order to get more details about port 134, let's try:

99)

```
┌──(root㉿INE)-[~]
└─# nmap -p 134 -sUV --script=default demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-25 13:46 IST
Nmap scan report for demo.ine.local (192.127.27.3)
Host is up (0.000031s latency).

PORT    STATE         SERVICE    VERSION
134/udp open|filtered ingres-net
MAC Address: 02:42:C0:7F:1B:03 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 113.22 seconds
```

Also try this one, specially when we don't know what service is (no version available):

```
└─# nmap demo.ine.local -p 134 -sUV --script=discovery
```

Unfortunately, this time the result it provides is the same as using the *default* script, nothing new.

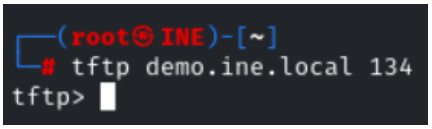
As a conclusion:

In one hand , services 177 and 234 are DNS and SNMP respectively. DNS because, despite not saying DNS explicitly, domain is equivalent to DNS!

On the other hand, Port 134/udp is open|filtered, meaning it may be open but not responding (very typical with UDP ports, not respond to generic probes), or it may be blocked by a firewall.

We have DNS, SNMP, so, according to the statement, the missing one is the TFTP, which by elimination, must be the 134.

to confirm it, we perform:



This manner, is verified!

LAB : SCAN THE SERVER III

Objectives of the lab: performing port scanning and service detection with Nmap (just like the earlier labs).

I performed:

```
# nmap -sS -A -p-  demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-25 14:54 IST
Nmap scan report for demo.ine.local (192.182.67.3)
Host is up (0.000058s latency).
All 65535 scanned ports on demo.ine.local (192.182.67.3) are in ignored states.
Not shown: 65535 closed tcp ports (reset)
MAC Address: 02:42:C0:B6:43:03 (Unknown)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop
```

As a result, thanks to the line, we can conclude that the entire TCP ports are closed :

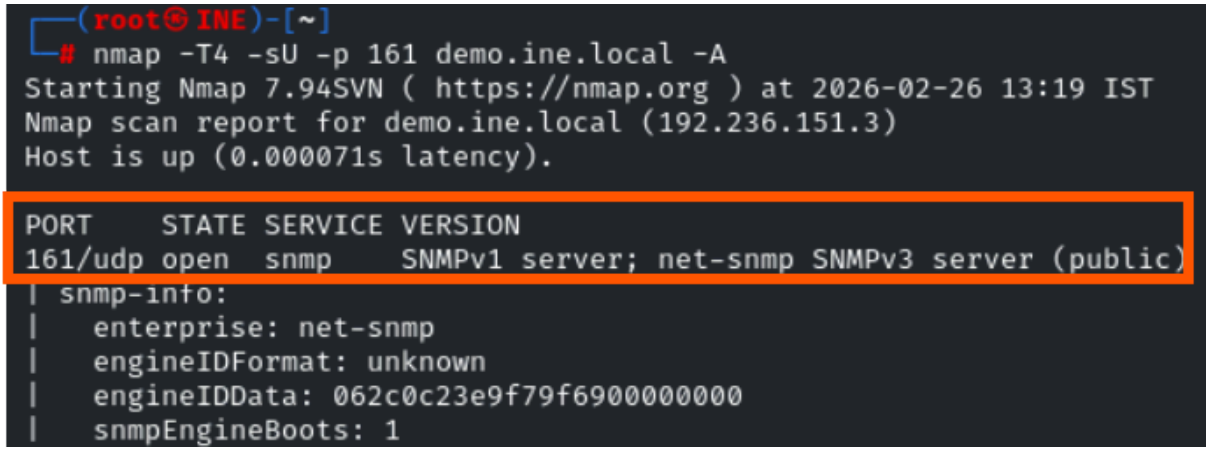
```
Not shown: 65535 closed tcp ports (reset)
```

Clarification: Not shown means: “I’m not showing them one by one because there are too many, but I’m summarizing their status”.

With -sS (a SYN scan), when Nmap reports closed (reset), it means the host replied with an RST packet — in other words, the port is reachable but closed (not filtered).

Next, we’ll try UDP ports. first we’ll try only the 1000 most common ports:

Once detected the port 161, I perform the following command, and the result is also displayed below:



LAB EXAM FOOTPRINTING & SCANNING

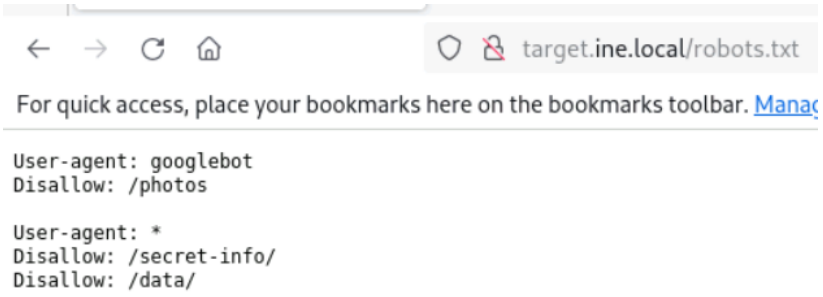
FLAG 1:

The server identifies itself when contacting with him:

```
┌──(root㉿INE)-[~]
└─# curl -I http://target.ine.local/
HTTP/1.1 200 OK
Server: Werkzeug/3.0.6 Python/3.10.12
Date: Fri, 27 Feb 2026 11:01:41 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 2557
Server: FLAG1_d74f0da6a9e345b5b025b1cdb3320a9a
Connection: close
```

FLAG 2:

When going to target.ine.local/robots.txt:



I suspect about secret-info, there can be the flag

I perform:

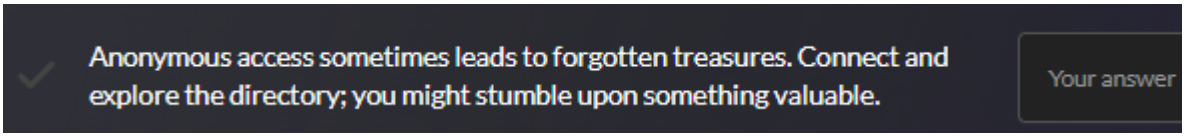
```
┌──(root㉿INE)-[~]
└─# curl http://target.ine.local/secret-info/
["flag.txt"]
```

Very important: this means flag.txt is within secret-info!

So, next, I perform:

```
┌──(root㉿INE)-[~]
└─# curl http://target.ine.local/secret-info/flag.txt
FLAG2_3ec389c5aebe483cb78f7288a50f3442
```

FLAG 3:



FTP (File Transfer Protocol) = un servicio para acceder a carpetas y archivos en un servidor web (ver, descargar y a veces subir).

Lo importante en CTF:

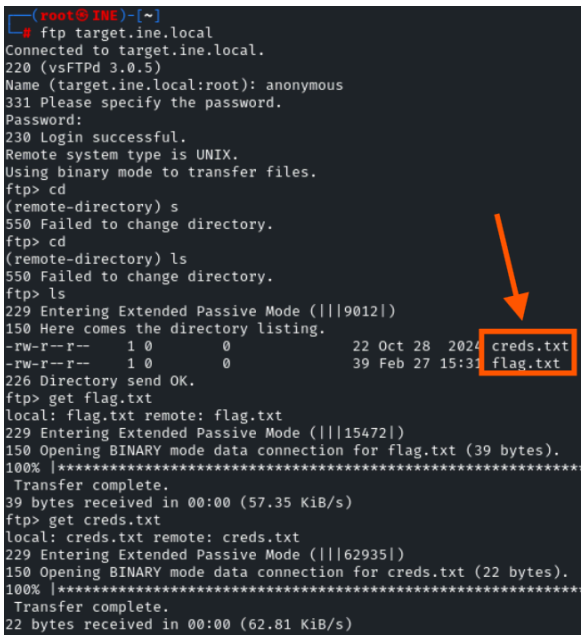
- Puede tener acceso anónimo (anonymous) → entras sin credenciales reales.
- Si entras, puedes explorar los directorios que el servidor expone (carpetas compartidas) y encontrar archivos "olvidados" como backups, configs o flag.txt (a veces incluso archivos de la web).

En un sitio como sport.es, si usaran FTP (muchos ya no, pero puede existir), por FTP normalmente se guardarían archivos del servidor, como:

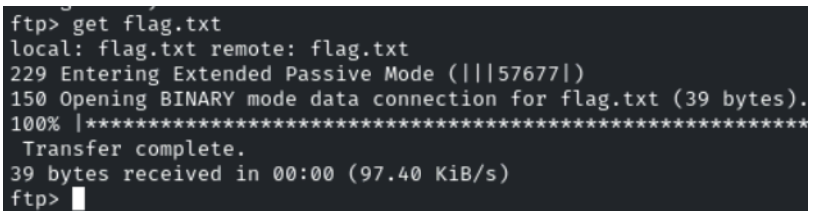
- Código/plantillas de la web o del CMS (PHP, plantillas, themes)
- Archivos estáticos: css/, js/, images/, iconos, PDFs, etc.
- Subidas: imágenes subidas desde el panel (a veces)
- Configuraciones: .env, config.php, wp-config.php (aquí a veces hay credenciales de BD)
- Backups: zips/tar con copias del sitio o de configuraciones

Que he hecho para capturar el flag:

1- en terminal:

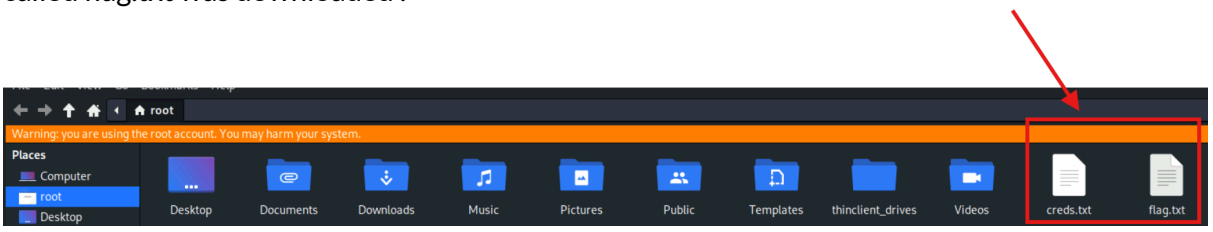


2- I performed:



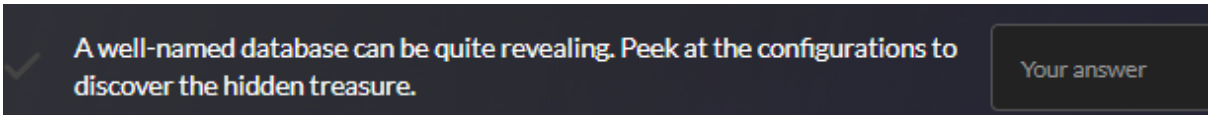
but any flag is displayed in the capture above.

The mistake i commit was not realising that , at the moment i performed that command, a file called flag.txt was downloaded :



When opening this file, we see the flag!

FLAG 4:



Database … this must remember us to MySQL. Also in the exercise clues, mentions MySQL.

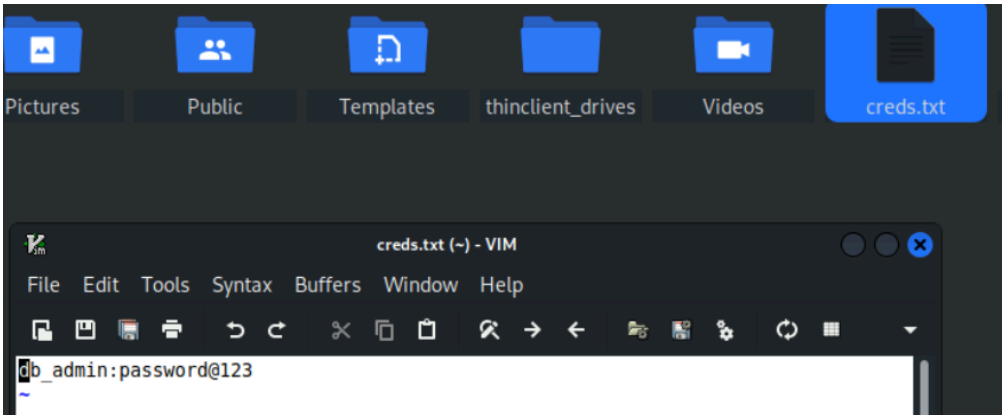
Whereas MySQL stores the data in tables (articles/news, users, comments) and usually only stores the path/URL to images, not the image file itself.

When entering in creds.txt, we see a user and a password. This must be used for sure to find the last flag.

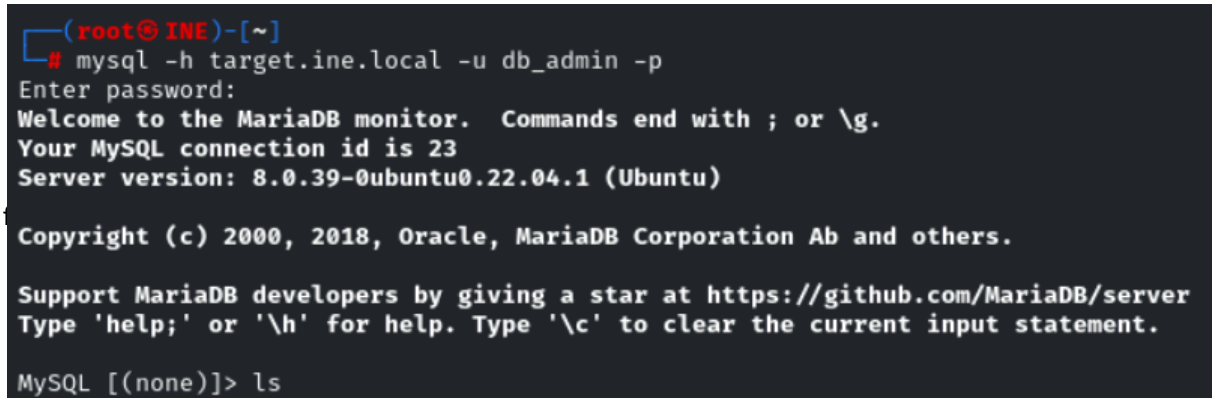
importante!!

user: db_admin

password : password@123

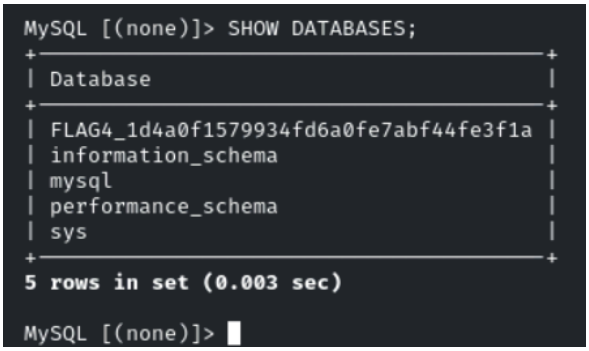


i log in to mysql:



Finally, i type a command used to see the databases:

MySQL → SHOW DATABASES:



SUB-COURSE : EVASION, SCAN PERFORMANCE & OUTPUT:

TO AVOID FIREWALLS AND IDS-IPS WHEN PERFORMING NMAP

Use -sA (ACK Scan, instead of Syn Scan)

For detect firewall is used the fragmentation!!

just we need to add the option -f

Can be used either `-f` or `--mtu`; they are alternative fragmentation options.

Also can be modified the packets by adding random data.

```
root@attackdefense:~# nmap -Pn -sS -sV -p445,3389 -f --data-length 200 -D 10.10.23.1
```

- -f (packet fragmentation)
  Makes Nmap send the probe packets split into smaller IP fragments instead of one full packet. The goal is to make it harder for some older/misconfigured firewalls or IDS to reassemble and recognize scan patterns.
  Note: modern devices usually reassemble fragments fine, so it's often still detected, and fragmentation can reduce scan reliability.

  Also to remark: A custom MTU lets me choose how small the fragments are instead of using Nmap's default fragmentation size. This can make detection harder for a basic or poorly configured IDS because the suspicious TCP information sent by Nmap is divided across several fragments instead of appearing together in one packet, as it would by default.

- --data-length 200
  Adds 200 bytes of random junk data to each probe packet, so the traffic doesn't match Nmap's typical packet sizes/signatures.
  Effect: more bandwidth/noise for the firewall and can affect timing; it doesn't "hide" the scan, it mostly changes how it looks.

- -D 10.10.23.1 (decoy)
  Sends the scan so it appears to come from multiple source IPs. This can confuse logs/analysts about which IP is the real scanner.
  Despite the src Ip changes, the MAC is the same so advanced firewalls can detect that and discover the trap.

  Key point: the decoy IP should be plausible/reachable in that network. Advanced monitoring can still correlate traffic and identify the real source.

  When capturing wireshark in my real eth , wireshark will show the decoy or the real one, depending on that packet!

To be even less suspicious, we can change the src port:

```
… 0.000… 10.10.23.4       10.4.27.83     IPv4    42 Fragmented IP protocol (proto=TCP 6, off=112, ID=839e) [Reassembled …
Transmission Control Protocol, Src Port: 33847, Dst Port: 3389, Seq: 0, Len: 200
  Source Port: 33847
```

performing the -g option:

```
root@attackdefense:~# nmap -Pn -sS -sV -p445,3389 -f --data-length 200 -g 53 -D 10.10.23.1,10.1
0.23.2 10.4.27.83
```

TIMINGS

The following host-timeout means: if in 5s doesn't respond, move on to the next host in the network.

```
root@attackdefense:~# nmap -Pn -sS -F --host-timeout 5s  10.10.23.0/24
```

The following is used to delay every paquet, so separating the packets 5 seconds between one and another.

This is a good way to look less suspicious, just because by default, when performing nmap packets, those are very consecutive and have a very little gap between them (miliseconds or less).

```
root@attackdefense:~# nmap -sS -sV -F --scan-delay 5s 10.4.26.5
```

Apply a display filter ... <Ctrl-/>

No.	Time	Source
6	19:46:33.664250661	02:42:0a:0a:17:
7	19:46:33.668201679	02:42:0a:0a:17:
8	19:46:33.668225199	02:42:11:18:2a:
9	19:46:38.564255958	10.10.23.2
10	19:46:38.574500808	10.4.26.5
11	19:46:38.574531159	10.10.23.2

Note: if needed, is nice to add the Time column! To see the time before and after the new configuration.

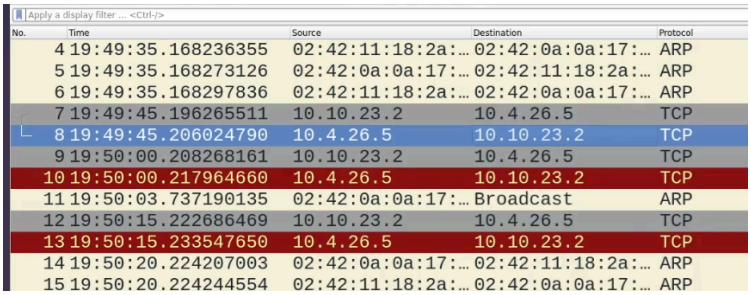
Is much more stealthier using T1 than T4 , for instance.

T1 makes the nmap too much time to perform and the time between the packets, so doesn't look at all like suspicious traffic!

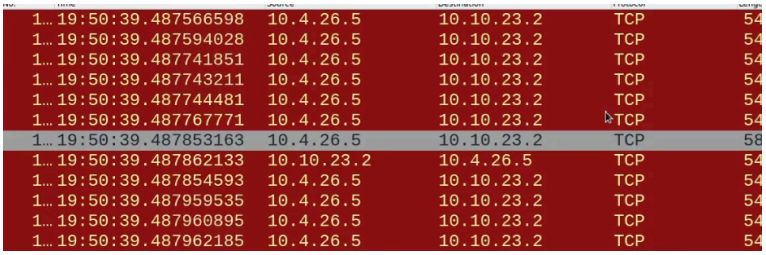
look:

```
root@attackdefense:~# nmap -sS -sV -F -T1 10.4.26.5
```

The result, as observed below: lot of time between every packet!



In the other case, if we perform with T4, the result in wireshark really stands out (very consecutive , less than milliseconds of distance):



NMAP OUTPUT FORMAT S

Next, is shown how to save in a normal format this nmap output:

```
root@attackdefense:~# nmap -Pn -sS -F -T4 10.4.19.132 -oN nmap_normal.txt
```

Xml format :

```
root@attackdefense:~# nmap -Pn -sS -F -T4 10.4.19.132 -oX nmap_xml.xml
```

Next, to be able to store in msf database, we need to activate postgresql service. If not, any nmap scan, xml, nor nothing will be successfully stored.

```
root@attackdefense:~# service postgresql start
Starting PostgreSQL 13 database server: main.
root@attackdefense:~# msfconsole
[*] StarTing the Metasploit Framework console...|
```

Next, do all this steps:

```
msf6 > workspace -a pentest_1
[*] Added workspace: pentest_1
[*] Workspace: pentest_1
msf6 > workspace
  default
* pentest_1
msf6 > db_status
[*] Connected to msf. Connection type: postgresq
msf6 > db_import nmap_xml.xml
[*] Importing 'Nmap XML' data
[*] Import: Parsing with 'Nokogiri v1.10.10'
[*] Importing host 10.4.19.132
[*] Successfully imported /root/nmap_xml.xml
msf6 >
```

Also useful:

We can see the address we scanned with nmap:

```
msf6 > hosts

Hosts
=====

address       mac  name  os_name  os_flavor  os_sp  purpose  info  comments
-------       ---  ----  -------  ---------  -----  -------  ----  --------
10.4.19.132                Unknown                   device
```

Very great, to see all the ports sorted.

```
address       mac  name  os_name  os_flavor  os_sp  purpose  info  comments
-------       ---  ----  -------  ---------  -----  -------  ----  --------
10.4.19.132                Unknown                   device

msf6 > services
Services
========

host          port   proto  name           state  info
----          ----   -----  ----           -----  ----
10.4.19.132   135    tcp    msrpc          open
10.4.19.132   139    tcp    netbios-ssn    open
10.4.19.132   445    tcp    microsoft-ds   open
10.4.19.132   3389   tcp    ms-wbt-server  open
10.4.19.132   49152  tcp    unknown        open
10.4.19.132   49153  tcp    unknown        open
10.4.19.132   49154  tcp    unknown        open
```

```
msf5 > db_nmap -Pn -sV -O 10.4.22.173
```

Very useful, as it will be saved in a very clear and visual format:

If I perform db_nmap, it will be executed the nmap scan immediately and saved automatically in the msf (Metasploit framework).

LAB: Windows Recon: SMB Nmap Scripts

Step 1 -

```
┌──(root㉿INE)-[~]
└─# nmap demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-28 21:19 IST
Nmap scan report for demo.ine.local (10.2.16.112)
Host is up (0.0031s latency).
Not shown: 992 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
49152/tcp open  unknown
49153/tcp open  unknown
49154/tcp open  unknown
49155/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 1.44 seconds
```

By the services names shown scanned with nmap, the target is very likely a Windows host.

Step 2- by searching in internet, the SMB default port is 445. Knowing that:

```
┌──(root㉿INE)-[~]
└─# ls -al /usr/share/nmap/scripts/ | grep -e "smb"
-rw-r--r-- 1 root root  3753 Jun 20  2024 smb2-capabilities.nse
-rw-r--r-- 1 root root  2689 Jun 20  2024 smb2-security-mode.nse
-rw-r--r-- 1 root root  1408 Jun 20  2024 smb2-time.nse
```

EXERCISE 1 LAB:

1. Identify SMB Protocol Dialects

So I search the smb scripts about the protocol:

```
┌──(root㉿INE)-[~]
└─# ls -al /usr/share/nmap/scripts/ | grep -e "smb" | grep -e "protocol"
-rw-r--r-- 1 root root  1833 Jun 20  2024 smb-protocols.nse
```

Step 3-

```
┌──(root㉿INE)-[~]
└─# nmap -sS -sV --script=smb-protocols -p 445 demo.ine.local
```

```
┌──(root㉿INE)-[~]
└─# nmap -sS -sV --script=smb-protocols -p 445 demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-28 21:31 IST
Nmap scan report for demo.ine.local (10.2.16.112)
Host is up (0.0031s latency).

PORT    STATE SERVICE      VERSION
445/tcp open  microsoft-ds Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
Service Info: OS: Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows

Host script results:
| smb-protocols:
|   dialects:
|     NT LM 0.12 (SMBv1) [dangerous, but default]
|     2:0:2
|     2:1:0
|     3:0:0
|_    3:0:2

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.38 seconds

┌──(root㉿INE)-[~]
```

Step 4-

EXERCISE 2 LAB:

2. Find SMB security level information

I do exactly the same as Exercise 1

Step 5 -

EXERCISE 2 LAB:

3. Enumerate active sessions, shares, Windows users, domains, services, etc.

I do exactly the same as Exercise 1 and 2:

```
┌──(root㉿INE)-[~]
└─# nmap -sS -sV --script=smb-enum-sessions demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-01 12:38 IST
Nmap scan report for demo.ine.local (10.2.23.52)
Host is up (0.0028s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE            VERSION
135/tcp   open  msrpc              Microsoft Windows RPC
139/tcp   open  netbios-ssn        Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds       Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
3389/tcp  open  ssl/ms-wbt-server?
49152/tcp open  msrpc              Microsoft Windows RPC
49153/tcp open  msrpc              Microsoft Windows RPC
49154/tcp open  msrpc              Microsoft Windows RPC
49155/tcp open  msrpc              Microsoft Windows RPC
49156/tcp open  msrpc              Microsoft Windows RPC
Service Info: OSs: Windows, Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows

Host script results:
| smb-enum-sessions:
|   Users logged in
|_    WIN-OMCNBKR66MN\bob since <unknown>
```

only queried SMB for session info using the smb-enum-sessions script. Because the server allows guest/anonymous (no-credentials) access, it revealed that there is an active session for:

- WIN-OMCNBKR66MN\bob

That means someone (or some process) is currently logged in as "bob" on the target, and Nmap was able to see/report it.

To actually "log in as bob," you would need bob's credentials (password/hash/ticket) and then authenticate to a service (SMB, WinRM, RDP, etc.). The scan output alone doesn't log you in.

If Guest login was disabled → we must authenticate (use a real username/password) and then run the same check to see active SMB sessions.

Step 5.2: run the shares script :

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-shares demo.ine.local
```

A share (in SMB/Windows) is a network-shared resource—usually a folder (or a special resource) that other machines can access over the network using a path like:

\\IP\SHARE_NAME

Examples from your output:

- \\10.2.23.52\Documents → a shared folder
- \\10.2.23.52\C$ → an administrative (hidden) share. It's hidden but it can still be accessed if you know that C$ exists!

However, we cannot access likewise, because it specifies the access as <none> (no way to access):

```
\\10.2.23.52\C$:
  Type: STYPE_DISKTREE_HIDDEN
  Comment: Default share
  Anonymous access: <none>
  Current user access: <none>
```

- \\10.2.23.52\IPC$ → a special share for inter-process communication (not a normal folder)

However, this share is also hidden, but we can still access it because read/write permissions are enabled:

```
\\10.0.30.254\IPC$:
  Type: STYPE_IPC_HIDDEN
  Comment: Remote IPC
  Anonymous access: <none>
  Current user access: READ/WRITE
```

In short: a share is the published name Windows exposes over SMB so you can access folders, printers, or special resources remotely.

Step 5.3: same as 5.2 but now using user and password

I perform:

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-shares --script-args smbusername=administrator,smbpassword=smbserver_771 demo.ine.local
```

Having logged-in with user and password, lots of directories (like ADMIN$, C$, D$) this time have the Read and Write option enabled! Just because without the guest log-in (5-2) has some limitations regarding this aspect (step 5.2).

Note: – script-args is used to enter the user and the password!

Step 5.4 : explore the users

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-users --script-args smbusername=administrator,smbpassword=smbserver_771 demo.ine.loca
```

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-users --script-args smbusername=administrator,smbpassword=smbserver_771 demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-01 13:54 IST
Nmap scan report for demo.ine.local (10.2.27.118)
Host is up (0.0036s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds

Host script results:
| smb-enum-users:
|   WIN-OMCNBKR66MN\Administrator (RID: 500)
|     Description: Built-in account for administering the computer/domain
|     Flags:       Password does not expire, Normal user account
|   WIN-OMCNBKR66MN\bob (RID: 1010)
|     Flags:       Password does not expire, Normal user account
|   WIN-OMCNBKR66MN\Guest (RID: 501)
|     Description: Built-in account for guest access to the computer/domain
|_    Flags:       Password not required, Password does not expire, Normal user account

Nmap done: 1 IP address (1 host up) scanned in 4.30 seconds

┌──(root㉿INE)-[~]
```

We can see in the shot above there are 3 users accounts!

Step 5.5 : explore the users

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-groups --script-args smbusername=administrator,smbpassword=smbserver_771 demo.ine.local
```

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-groups --script-args smbusername=administrator,smbpassword=smbserver_771 demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-04 16:07 IST
Nmap scan report for demo.ine.local (10.0.30.254)
Host is up (0.0023s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds

Host script results:
| smb-enum-groups:
|   Builtin\Administrators (RID: 544): Administrator, bob
|   Builtin\Users (RID: 545): bob
|   Builtin\Guests (RID: 546): Guest
|   Builtin\Power Users (RID: 547): <empty>
|   Builtin\Print Operators (RID: 550): <empty>
|   Builtin\Backup Operators (RID: 551): <empty>
|   Builtin\Replicator (RID: 552): <empty>
|   Builtin\Remote Desktop Users (RID: 555): bob
|   Builtin\Network Configuration Operators (RID: 556): <empty>
|   Builtin\Performance Monitor Users (RID: 558): <empty>
|   Builtin\Performance Log Users (RID: 559): <empty>
|   Builtin\Distributed COM Users (RID: 562): <empty>
|   Builtin\IIS_IUSRS (RID: 568): <empty>
|   Builtin\Cryptographic Operators (RID: 569): <empty>
|   Builtin\Event Log Readers (RID: 573): <empty>
|   Builtin\Certificate Service DCOM Access (RID: 574): <empty>
|   Builtin\RDS Remote Access Servers (RID: 575): <empty>
|   Builtin\RDS Endpoint Servers (RID: 576): <empty>
|   Builtin\RDS Management Servers (RID: 577): <empty>
|   Builtin\Hyper-V Administrators (RID: 578): <empty>
|   Builtin\Access Control Assistance Operators (RID: 579): <empty>
|   Builtin\Remote Management Users (RID: 580): <empty>
|_  WIN-OMCNBKR66MN\WinRMRemoteWMIUsers__ (RID: 1000): <empty>
```

Because I provide Administrator credentials, the script is able to retrieve this information.

How to read the output:
Each line is:

Group (with its RID): members

- Builtin\Administrators: Administrator, bob → bob is a local administrator (high privileges).
- Builtin\Users: bob → bob is also in the default Users group (standard user group).
- Builtin\Guests: Guest → the Guest account exists and is in the Guests group.
- Builtin\Remote Desktop Users: bob → bob is allowed to log in via RDP (Remote Desktop), if the service is enabled (and your scan shows port 3389 is open).

Step 5.6 : Enumerating the services running on the target

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-services --script-args smbusername=administrator,smbpassword=smbserver_771 demo.ine.local
```

```
PORT    STATE SERVICE
445/tcp open  microsoft-ds
| smb-enum-services:
|   AmazonSSMAgent:
|     display_name: Amazon SSM Agent
|     state:
|       SERVICE_PAUSED
|       SERVICE_RUNNING
|       SERVICE_CONTINUE_PENDING
|       SERVICE_PAUSE_PENDING
|     type:
|       SERVICE_TYPE_WIN32
|       SERVICE_TYPE_WIN32_OWN_PROCESS
|     controls_accepted:
|       SERVICE_CONTROL_STOP
|       SERVICE_CONTROL_NETBINDENABLE
|       SERVICE_CONTROL_NETBINDADD
|       SERVICE_CONTROL_INTERROGATE
|       SERVICE_CONTROL_CONTINUE
|       SERVICE_CONTROL_PARAMCHANGE
|   DiagTrack:
|     display_name: Diagnostics Tracking Service
|     state:
|       SERVICE_PAUSED
```

Step 5.7 : Enum the shares folders

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-shares,smb-ls --script-args smbusername=administrator,smbpassword=smbserver_771 demo.ine.local
```

```
┌──(root㉿INE)-[~]
└─# nmap -p445 --script smb-enum-shares,smb-ls --script-args smbuserr
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-04 16:09 IST
Nmap scan report for demo.ine.local (10.0.30.254)
Host is up (0.0023s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds

Host script results:
| smb-ls: Volume \\10.0.30.254\ADMIN$
|   maxfiles limit reached (10)
| SIZE   TIME                 FILENAME
| <DIR>  2013-08-22T13:36:16  .
| <DIR>  2013-08-22T13:36:16  ..
| <DIR>  2013-08-22T15:39:31  ADFS
| <DIR>  2013-08-22T15:39:31  ADFS\ar
| <DIR>  2013-08-22T15:39:31  ADFS\bg
| <DIR>  2013-08-22T15:39:31  ADFS\cs
| <DIR>  2013-08-22T15:39:31  ADFS\da
| <DIR>  2013-08-22T15:39:31  ADFS\de
| <DIR>  2013-08-22T15:39:31  ADFS\el
| <DIR>  2013-08-22T15:39:31  ADFS\en
|
|
| Volume \\10.0.30.254\C
|   maxfiles limit reached (10)
| SIZE   TIME                 FILENAME
| <DIR>  2013-08-22T15:39:30  PerfLogs
```

smb-enum-shares lists all shares it can enumerate, including hidden ones like ADMIN$, C$, IPC$.

smb-ls then tries to list files inside each share.

LAB 2: WINDOWS RECONNAISSANCE WITH ZENMAP

Objective: discover all available hosts

step 1:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . : ap-southeast-1.compute.internal
   Link-local IPv6 Address . . . . . : fe80::1c7a:651c:35f4:2a45%4
   IPv4 Address. . . . . . . . . . . : 10.0.17.63
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . : 10.0.16.1
PS C:\Users\Administrator>
```

1st row : looks like its an internal network (typical from AWS), not important at all.

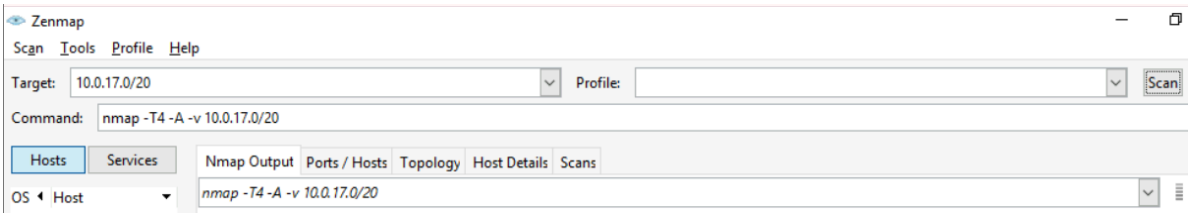
2nd row: Ip not important at all

3rd row: my private IP

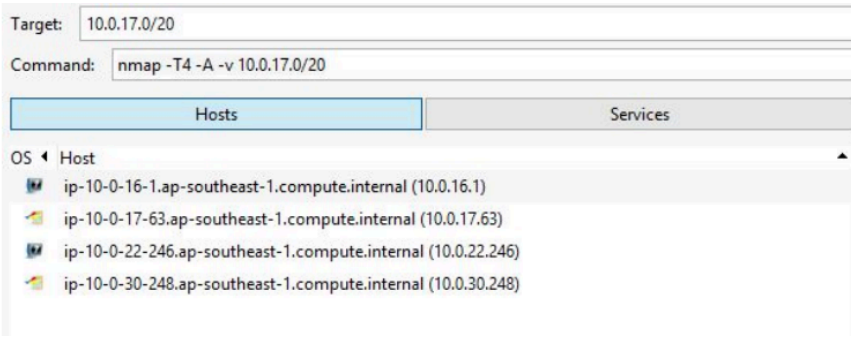
4th row: the mask of my subnet

5th row: default gw (is the router's IP i'll use to go to other subnetworks and internet).

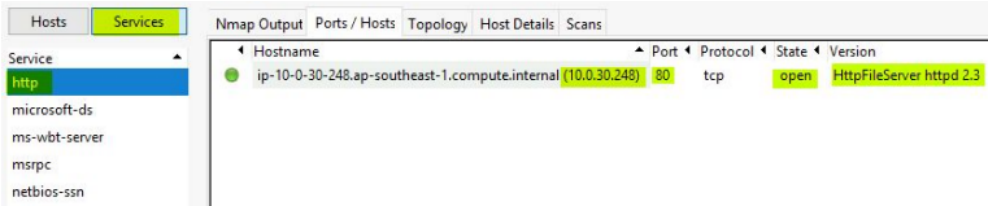
Now I open the zenmap app, and I click to scan:



The result is as follows:

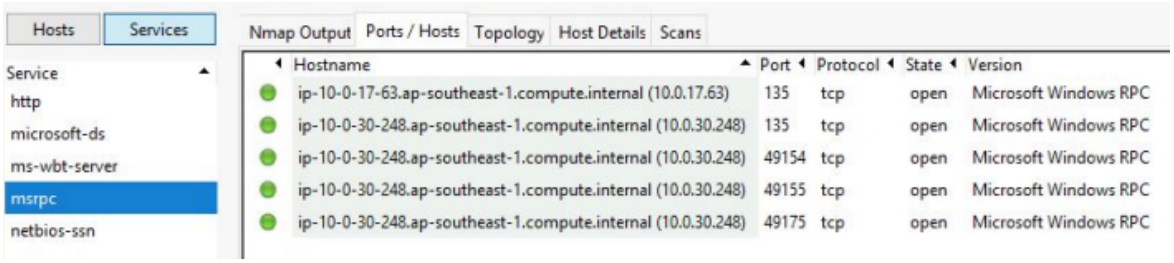


we detected this 4 hosts in the subnet (one of these is us, the .63).

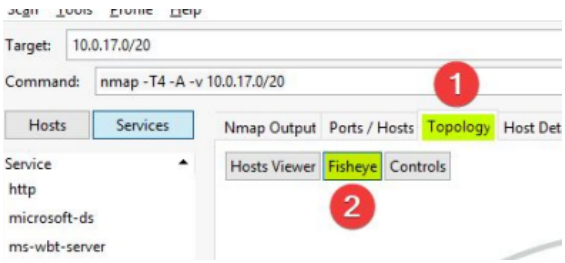


As seen, http (80) was only detected on port 248

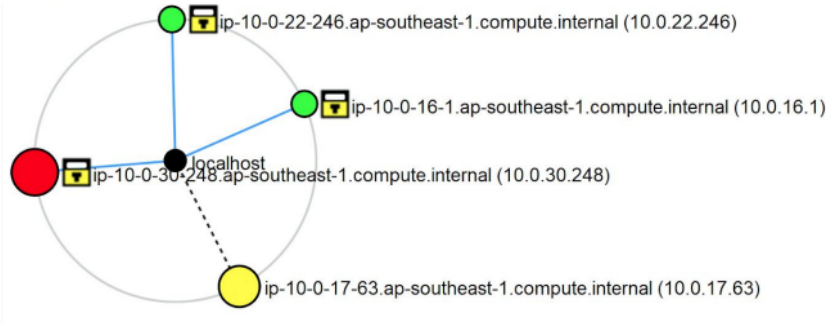
By contrast, msrpc service was detected in all of this ports. We can observe below, that host .248 has the msrpc service opened in 4 different ports.



To finish, if i click this 2 sections:



a diagram will be displayed:



Yellow is just us, who launched the attack. The yellow host is us, while the 2 green and the red are the 3 hosts discovered by zenmap. red means that that host is alive but is not responding / inaccessible (maybe ids/ips, maybe firewall,etc). We were able only to detect he's there, maybe thanks to the firewall or somebody who notified us it was alive.

The 2 green hosts indicate they are alive and reachable, we can establish a communication with them.

NOTE: BEFORE PORT SCANNING, OFTEN, DOESN'T POINT TO ANY CONCRETE PORT OF THE TARGET HOST. IF USES ARP OR ICMP PROTOCOLS, THAT ONES DON'T ALLOW TO USE PORTS. YOU TRY TO CONTACT THAT TARGET HOST, BUT NOT THROUGH PORTS.